Design and Implementation of a Digital IIR Filter with Floating-Point Representation

1st Fabio Luna
Department of Electrical Engineering
Federal University of Juiz de Fora
Juiz de Fora, Brazil
fabio.luna@estudante.ufjf.br

2nd Luciano Andrade
Department of Electrical Engineering
Federal University of Juiz de Fora
Juiz de Fora, Brazil
luciano.andrade@ufjf.br

3nd Thiago Paschoalin
Dept. Electroeletronic
Fed. Cent. for Technol. Educ. of Minas Gerais
Leopoldina, Brazil
thiago.paschoalin@cefetmg.br

***Abstract*—This paper presents the implementation of a digital IIR filter on an FPGA using floating-point operations to minimize errors caused by quantization. A fully combinational floating-point multiply-and-accumulate circuit is proposed, avoiding the use of pipeline stages typically required in conventional floating-point FPGA implementations. This architectural choice simplifies control, enables immediate output response, and is particularly advantageous in applications requiring low latency and predictable timing. The quantization process in fixed-point designs is highly specific to each project and often demands extensive tuning and simulation, significantly increasing development time. By adopting floating-point arithmetic, the proposed method eliminates this overhead, offering greater flexibility and accelerating the design process, albeit at the cost of increased logic utilization. The implementation aims to reduce deviations from the ideal impulse response while maintaining filter stability and achieving high numerical accuracy with fewer bits. The results demonstrate the effectiveness of this approach compared to traditional quantization-based methods, particularly in scenarios where design time and precision are critical.**

***Index Terms*—Digital filters, FPGA, floating point, quantization, signal processing.**

I. Introduction

Digital filters play a key role in signal processing, enabling manipulation of discrete-time signals. Among the main types, FIR filters exhibit a finite impulse response:

$$y[n] = \sum_{k=0}^{N-1} h[k]x[n-k] \tag{1}$$

IIR filters, by incorporating feedback from past outputs, exhibit an infinite impulse response and achieve similar frequency characteristics with fewer coefficients. However, they may introduce stability concerns:

$$y[n] = \sum_{i=0}^{N-1} b_i x[n-i] - \sum_{i=1}^{M-1} a_i y[n-i] \tag{2}$$

FPGA-based implementations typically rely on fixed-point arithmetic, requiring quantization that introduces errors relative to ideal software models [1], [2]. Increasing bit-width reduces these errors but raises logic complexity [9], [8]. Moreover, quantization tuning is highly application-specific and time-consuming [7].

This work proposes an IIR filter implemented on an FPGA using floating-point arithmetic through fully combinational circuits. The proposed architecture avoids pipelining, enabling immediate output propagation and simplified control logic [6], [5]. Although this increases combinational complexity and may reduce maximum frequency, it eliminates quantization errors and significantly shortens design time [8], [9], [7]. In addition, real-time IIR implementations typically avoid pipelined architectures due to their recursive nature and feedback timing constraints [12], [13].

II. IIR Filter

As shown in (2), IIR filters compute each output based on current and past inputs as well as past outputs, which characterizes their feedback nature. In the Z-domain, the general transfer function is given by:

$$H(z) = \frac{P(z)}{D(z)} = \frac{p_0 + p_1 z^{-1} + \ldots + p_M z^{-M}}{d_0 + d_1 z^{-1} + \ldots + d_N z^{-N}} \tag{3}$$

Assuming M=N=3, the third-order IIR filter can be expressed as the product of two components:

$$H_1(z) = \frac{W(z)}{X(z)} = P(z) = p_0 + p_1 z^{-1} + p_2 z^{-2} + p_3 z^{-3} \tag{4}$$

$$H_2(z) = \frac{Y(z)}{W(z)} = \frac{1}{D(z)} = \frac{1}{1 + d_1 z^{-1} + d_2 z^{-2} + d_3 z^{-3}} \tag{5}$$

These blocks form the Direct Form I structure, illustrated in Figure 1. An alternative configuration, Direct Form II, is shown in Figure 2 [3].

Other implementations, such as cascade and parallel forms, are also used, but Direct Form I is adopted in this work [1], [3].

III. Floating-Point Representation

Digital circuits typically operate using integer arithmetic. To support floating-point operations, values must be encoded in a format that enables manipulation as integers [8].

This work adopts a format based on the IEEE 754 standard [2], allowing a wide dynamic range and scalable precision

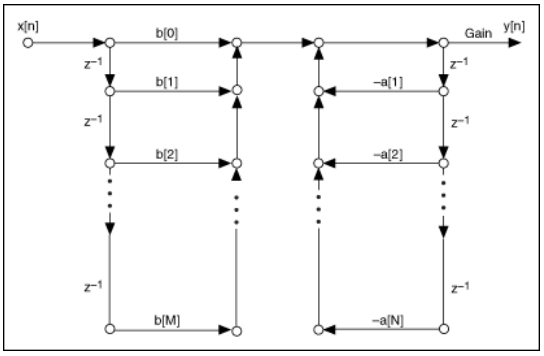

Fig. 1. Direct Form I

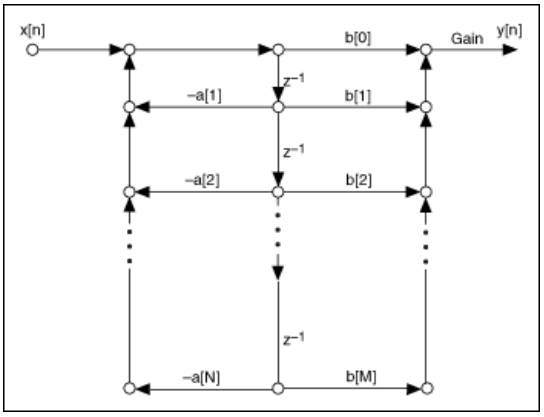

Fig. 2. Direct Form II

depending on the bit-width used [9]. Although the standard defines 32-bit single-precision representation, this structure can be adapted to reduced widths for FPGA implementations.

The floating-point number is divided into three fields: sign, exponent, and mantissa. In the 32-bit format, 1 bit is allocated for the sign, 8 bits for the exponent, and 23 bits for the mantissa. The numerical value is computed as:

$$N_{float} = (-1)^s \times M \times 2^{\text{Exp}} \tag{6}$$

where s is the sign bit, M is the mantissa, and Exp is the exponent in two's complement.

For example, the number 1.1 is encoded as:

$$0\ 11101010\ 10001100110011001100111$$

- $s = 0$
- $\text{Exp} = 11101010_2 = -22$
- $M = 4{,}613{,}735$

Substituting into (6):

$$N_{float} = (-1)^0 \times 4{,}613{,}735 \times 2^{-22} \approx 1.100000143 \tag{7}$$

This yields an absolute error of approximately 1.4×10^{-7}, illustrating the high precision attainable with 32-bit representation.

IV. Implemented Circuit

The circuit was implemented using *Quartus*, and the overall block diagram is shown in Fig. 3. The project consists of three main stages: impulse generation and conversion to floating-point; application of the IIR filter using custom arithmetic circuits; and data extraction for result comparison [6], [9].

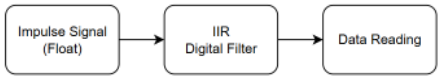

Fig. 3. Overall block diagram of the project

A. Developed Digital Circuits

The key circuits developed include normalization, integer-to-floating-point conversion, addition, and multiplication.

1) Normalization: As shown in Fig. 4, normalization adjusts the mantissa through multiplexed shifts and updates the exponent accordingly (Fig. 5).

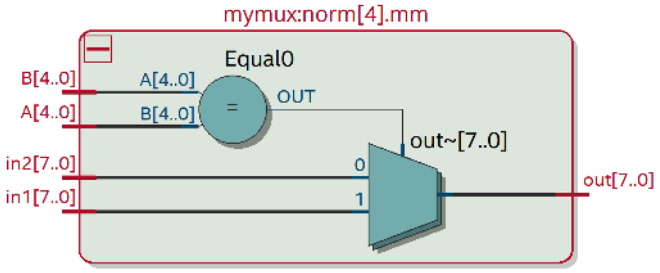

Fig. 4. Multiplexer for normalization

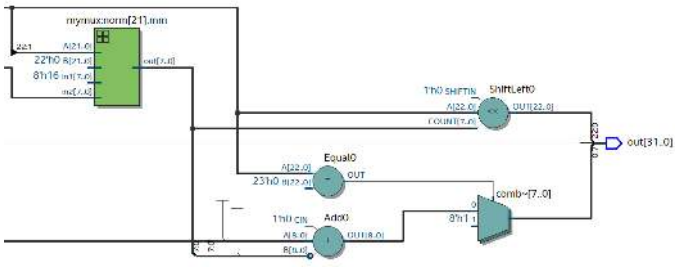

Fig. 5. Bit shift of the exponent

2) Integer to Floating-Point Conversion: The circuit in Fig. 6 extracts the sign, adjusts the mantissa, initializes the exponent, and applies normalization.

3) Addition Circuit: The adder (Fig. 7) includes denormalization to align exponents, followed by arithmetic and normalization stages [2], [8].

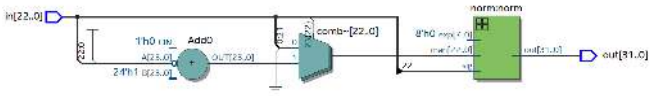

Fig. 6. Integer to floating-point conversion

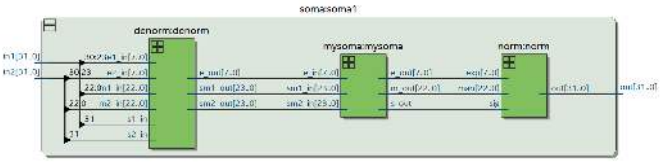

Fig. 7. Floating-point addition circuit

4) Multiplication Circuit: Fig. 8 shows the multiplication process: unpacking, computing product, and normalizing the result as in Eq. (8).

$$N = n_1 \times n_2 = (M_1 M_2) \cdot 2^{(\exp_1 + \exp_2)} \tag{8}$$

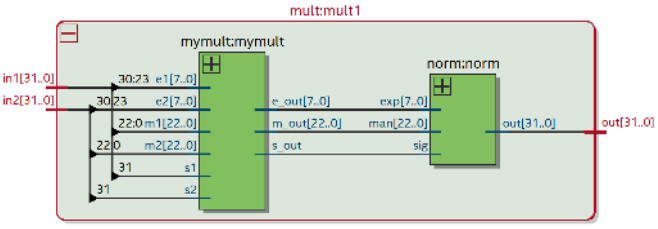

Fig. 8. Floating-point multiplication circuit

B. Impulse Generation and Filter Application

The impulse input is generated and converted using the circuit in Fig. 6. The IIR filter, implemented in Direct Form I, uses 8th-order coefficients, also represented in the custom floating-point format. A *Python* script was used for coefficient conversion [1].

C. Data Reading

Output values, still in custom format, are read and converted to standard *float* using a script. The process is illustrated in Fig. 9. To evaluate error, an ideal response was generated using `scipy`'s `lfilter` [1]. This process was repeated across bit-width configurations listed in Table I.

V. Results

The output of the implemented filter was compared to the ideal impulse response to evaluate numerical accuracy. The absolute error is defined as:

$$e_{abs} = y_{ideal} - y_{digital} \tag{9}$$

The final error is computed as the ratio between the energy of the error and the energy of the ideal signal:

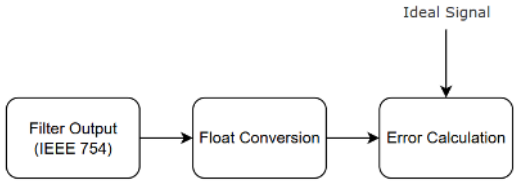

Fig. 9. Data reading and error evaluation

Total Bits	Exponent Bits	Mantissa Bits
12	7	4
14	7	6
16	9	6
18	10	7
20	11	8
24	15	8
28	19	8
32	23	8

TABLE I
BIT-WIDTH CONFIGURATIONS USED

$$e_{final} = \frac{\sum e_{abs}^2}{\sum y_{ideal}^2} \tag{10}$$

This metric provides a normalized assessment of filter accuracy and is widely adopted in DSP applications [1]. The target is to keep the error below 1%. Fig. 10 shows the error as a function of bit-width.

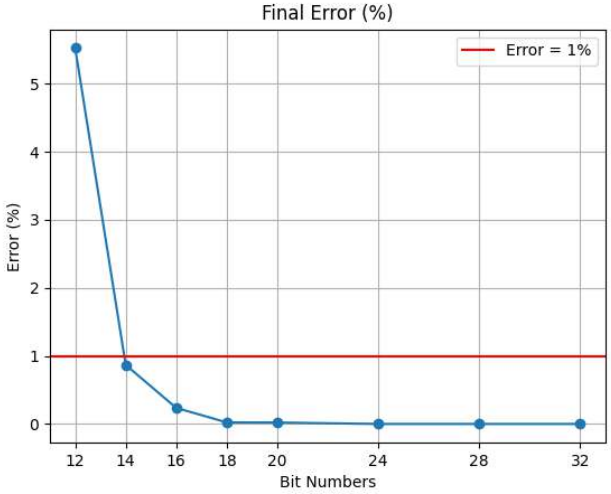

Fig. 10. Final error versus total number of bits

As expected, higher bit-widths yield lower errors. From 14 bits onward, the error drops below 1%, reaching negligible levels at 32 bits. With only 12 bits, the error exceeds 5%.

Table II summarizes the error for each configuration.

For comparison, a conventional fixed-point IIR filter with 38-bit datapath and 17-bit output yields an error of $8.71 \times 10^{-4}\%$. Reducing bit-width further causes the error to surpass the 1% threshold [6], [9].

Total Bits	Error (%)
12	5.53
14	0.87
16	0.24
18	2.32×10^{-2}
20	2.14×10^{-2}
24	4.05×10^{-5}
28	1.61×10^{-7}
32	6.08×10^{-10}

TABLE II
FINAL ERROR FOR EACH BIT-WIDTH CONFIGURATION

The floating-point filter with 24-bit datapath offers better accuracy, but at higher hardware cost. It consumed 5% of FPGA logic (2,797 out of 56,480), 133 registers, and 16 DSP blocks, as shown in Fig. 12. The fixed-point version used less than 1% of logic, 199 registers, and 9 DSP blocks (Fig. 11) [8], [7].

Logic utilization (in ALMs)	113 / 56,480 (< 1 %)
Total registers	199
Total pins	28 / 268 (10 %)
Total virtual pins	0
Total block memory bits	0 / 7,024,640 (0 %)
Total DSP Blocks	9 / 156 (6 %)
Total HSSI RX PCSs	0 / 6 (0 %)
Total HSSI PMA RX Deserializers	0 / 6 (0 %)
Total HSSI TX PCSs	0 / 6 (0 %)
Total HSSI PMA TX Serializers	0 / 6 (0 %)
Total PLLs	0 / 13 (0 %)
Total DLLs	0 / 4 (0 %)

Fig. 11. Logic utilization for fixed-point IIR filter

Logic utilization (in ALMs)	2,797 / 56,480 (5 %)
Total registers	133
Total pins	40 / 268 (15 %)
Total virtual pins	0
Total block memory bits	0 / 7,024,640 (0 %)
Total DSP Blocks	16 / 156 (10 %)
Total HSSI RX PCSs	0 / 6 (0 %)
Total HSSI PMA RX Deserializers	0 / 6 (0 %)
Total HSSI TX PCSs	0 / 6 (0 %)
Total HSSI PMA TX Serializers	0 / 6 (0 %)
Total PLLs	0 / 13 (0 %)
Total DLLs	0 / 4 (0 %)

Fig. 12. Logic utilization for floating-point implementation

VI. CONCLUSION

This work presented the implementation of a digital IIR filter using floating-point arithmetic, eliminating the need for coefficient quantization. This approach serves as an alternative to conventional methods, given that digital circuits typically operate using fixed-point arithmetic.

By avoiding quantization, the proposed method reduces numerical errors and provides higher precision in the filter response, while preserving system stability. Moreover, it achieves this accuracy with fewer bits compared to traditional fixed-point implementations.

On the other hand, a significant increase in computational effort was observed during FPGA synthesis, as the logic resource usage was substantially higher than that of the conventional filter implementation (approximately 10% for floating-point, compared to less than 1% for fixed-point). However, unlike most floating-point implementations that rely on pipelined architectures to manage complexity and latency, the proposed design uses a fully combinational approach. This simplifies control logic and eliminates the latency introduced by pipeline stages, offering a deterministic and immediate output response.

In summary, the proposed circuit offers a notable gain in precision, requiring fewer bits to achieve the same error levels. While this increases hardware usage, it also simplifies the design by eliminating quantization analysis—an increasingly viable trade-off given the growing capacity of modern FPGAs.

REFERENCES

[1] S. K. Mitra, *Digital Signal Processing: A Computer-Based Approach*, 3rd ed. New York: McGraw-Hill, 2005.

[2] *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019 (Revision of IEEE 754-2008), pp. 1–84, 2019. DOI: 10.1109/IEEESTD.2019.8766229.

[3] National Instruments, "IIR Direct Form Structures," https://www.ni.com/docs/en-US/bundle/labview-digital-filter-design-toolkit-api-ref/page/lvdfdtconcepts/iir_direct_form_structures.html, Accessed: Mar. 14, 2025.

[4] V. A. M. Santos, "Implementação de circuitos aritméticos em ponto flutuante, utilizando formato com número de bits configurável," TCC, Universidade Federal de Juiz de Fora, Juiz de Fora, MG, 2017.

[5] M. Garrido, J. Grajal, M. A. Sánchez, and O. Gustafsson, "Pipelined architectures for high-speed FIR filters using look-ahead techniques," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 60, no. 7, pp. 1675–1686, 2013.

[6] C. Lee and W. Luk, "Design and evaluation of a customizable floating-point multiply-accumulate unit for FPGAs," in *Proc. IEEE FCCM*, 2019, pp. 177–180.

[7] I. Kuon and J. Rose, "Measuring the gap between FPGAs and ASICs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 26, no. 2, pp. 203–215, 2007.

[8] M. Dupuis and B. Ulicny, "Hardware floating-point units in FPGA-based real-time systems: Performance and design considerations," in *Proc. IEEE ICIT*, 2018, pp. 1412–1417.

[9] J. D. Bakos, "FPGA acceleration of numerical algorithms with high-level synthesis," *IEEE Transactions on Computers*, vol. 61, no. 10, pp. 1370–1377, 2012.

[10] J. Zhu, W. Lin, J. Wang, and Q. Zhang, "A configurable floating-point IIR filter design for FPGA-based applications," in *Proc. IEEE Int. Symp. Circuits and Systems (ISCAS)*, 2015, pp. 1642–1645.

[11] M. Ayinala and J. Choi, "Floating-point based digital filter implementation in FPGAs," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 57, no. 6, pp. 446–450, 2010.

[12] S. Roy, A. Gupta, and P. K. Meher, "A Parallel Pipelined Architecture for High-Speed Real-Time IIR Filter Implementation," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 67, no. 5, pp. 1597–1607, May 2020.

[13] H. Ben Abdallah and A. Baganne, "Real-Time Implementation Constraints of IIR Filters on FPGA: An Evaluation Study," in *Proceedings of the 2021 International Conference on Electronics, Control, Optimization and Computer Science (ICECOCS)*, 2021, pp. 1–6.